

Full Stack Development with MERN

Project Documentation format

1. Introduction

• **Project Title:** Customer Registry

• **Team Members:**

Team ID: SWTID-2026-9658

Core Engineering & Full-Stack Development:

- Mounika Chittoor (Team Lead)
- Monika Dondeti
- Meghana Siddamurthy
- Gummagatta Naveen Kumar
- Kariyappagari Manjunatha

2. Project Overview

• **Purpose:** The purpose of the Customer Care Registry is to provide a clean, secure, and fast platform where customers can easily log complaints and interact directly with support agents to resolve technical issues or operational grievances.

• **Features:**

- Secure user signup, login authentication, and state management.
- Single-page multi-parameter forms for creating customer tickets.
- Role-based visibility displaying a customer-specific complaint history track and a centralized administrator department queue.
- Seamless communication history channels powered by text-based messaging between customers and support executives.

3. Architecture

- **Frontend:** A modern Single Page Application (SPA) designed using React 19 and Vite. It leverages React Client-Side Routing for transition layers and utilizes Context Providers to enforce global session authentication rules across pages
- **Backend:** Built using Node.js and the Express.js v5 web routing framework. The server handles application routing middleware logic, processes asynchronous requests, prints operational requests to log parameters, and implements global error handling.
- **Database:** Powered by a non-relational MongoDB Document Database framework managed via the Mongoose Object-Document Mapper (ODM).

Data models define robust validations for users, complaints, and interaction messages

4. Setup Instructions

• **Prerequisites:** Node.js runtime environment installed locally.
MongoDB Community Server local instance or an active cloud cluster connection (MongoDB Atlas).

Installation: Clone the project repository:

<https://github.com/chittoormounika0309-ops/Customer--Registry>

set up Backend Dependencies:

Bash

cd server

npm install

Configure Environment Variables: Create a `.env` file inside your `/server` directory:

Code snippet

PORT=5000

MONGO_URI=your_mongodb_connection_string

JWT_SECRET=your_jwt_encryption_key

Set up Frontend Dependencies:

Bash

cd ../client

npm install

5. Folder Structure

• Client:

Customer---Care/

├── client/ # React Frontend Application (Vite Build)

— public/	# Static assets (favicons, icons)
— src/	# Component Logic & Views
— context/	# AuthContext global user session data
— pages/ etc.)	# View Screens (Home, Login, SignUp, WriteComplaint, etc.)
— App.jsx	# Client-side route declarations
— package.json	# Frontend package dependencies & scripts
— vite.config.js	# Vite build environment configuration
— server/	# Node.js Backend Application
— controllers/	# Request handler processing logic
— models/	# Mongoose schemas (User, Complaint, Message)
— routes/	# Express API endpoint mapping modules
— db.js	# Database connectivity setup code
— server.js	# Application middleware stack configuration
— index.js	# System bootstrapping & server network listener
— package.json	# Backend configuration & reliance scripts

6. Running the Application

- Provide commands to start the frontend and backend servers locally.

◦ **Frontend:** `cd client npm run dev`

◦ **Backend:** `cd server npm start`

7. API Documentation

All core routes map directly to exposed JSON API nodes:

- **POST /api/auth/signup:** Registers a new system account securely using bcryptjs hashing.
- **POST /api/auth/login:** Authenticates credentials and returns a secure authorization token signature.

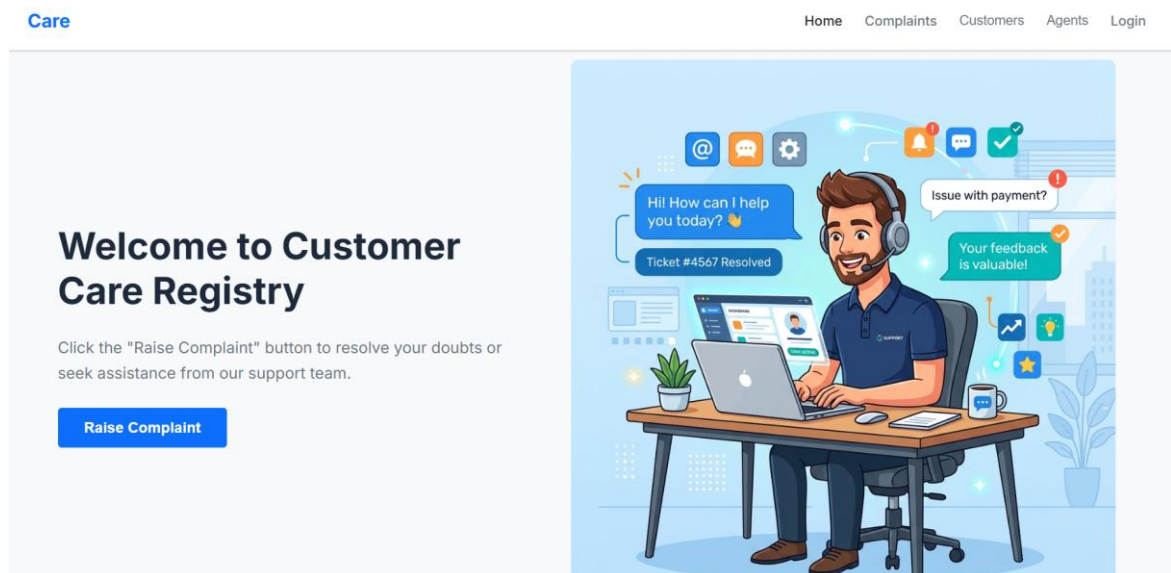
- **GET /api/complaints:** Fetches registered customer care logs based on individual or agent credential flags.
- **POST /api/complaints:** Submits a valid ticket layout record container directly into the database.
- **POST /api/messages:** Publishes synchronous support messaging conversations to the user dashboard queue.

8. Authentication

• Authentication is securely completed via stateless JSON Web Tokens (JWT) combined with backend **Bcrypt password encryption fields**. Upon validating accurate logins, the server builds an encrypted signature token and returns it. The React layout layer captures this value, updates the dynamic state inside the application's root via <AuthProvider>, and securely appends the authorization header into all future outgoing requests.

9. User Interface

Home Page:



Login Page:

Care

HomeComplaintsCustomersAgentsLogin

Login

Email

Enter email

Password

.....

Login

Don't have an account? [Sign Up](#)

Registration Page:

Care

HomeComplaintsCustomersAgentsLogin

Sign Up

First Name

Enter first name

Last Name

Enter last name

User Name

Enter user name

Email

Enter email

Password

Enter password

Type (admin, user, agent)

Enter type (admin, user, agent)

Sign Up

Complaints page:

Care

Home ChatWithAgent MyComplaints Logout Dropdown



Write Your Complaint

Name

Phone

Email

Complaint Details

Describe your complaint here...

Submit

My Complaints page:

Care

Home ChatWithAgent MyComplaints Logout Dropdown

My Complaints

ID: mock_comp_1784053473896awj58

Complaint: Login issue

Date: 7/14/2026

Status: Pending

Agent DashBoard:

Gifts

Home Logout Dropdown

CHAT INFO

C Mounika

C Mounika

Complaint Details

Total	Solved	Pending
1	0	1

No messages yet. Start the conversation!

Type a message

10. Testing

- Backend endpoint verification is performed locally using API evaluation utilities to track dynamic payload requests and ensure that response status formats remain accurate under error testing scripts.
- Frontend build pipelines are controlled via structural code evaluations using structural code compilation tools (oxlint) to prevent dead code blocks or performance layout errors.

11. Screenshots or Demo

•**Repository Link:** <https://github.com/chittoormounika0309-ops/Customer-->

Registry

12. Known Issues

- •Currently, the background execution architecture handles connections safely; however, missing environment configuration parameters will trigger a manual database fallback logic alert within the system connection window during initial local initialization phases.

13. Future Enhancements

- Adding support for localized dynamic configuration grids enabling customized input parameters for user tickets.
- Introducing physical interactive communication registries allowing customer service executives to trace telephone conversation logs or scheduled physical review appointments manually from the admin control panel.